K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

Program- B.Tech (AI/ML)

Course Title- Data Structures

Course Code- ETCCDS202

Unit- Foundations & Algorithmic Analysis

Student Name- Adriana Aharyan Roll no.2501730153

Submission Date-04.03.2026 Section- A

Part A: Stack ADT (ADT + Operations)

1.What is an ADT?

An Abstract Data Type (ADT) is a logical model of a data structure that defines what operations can be performed but does not specify how they are implemented. It focuses on behavior, not implementation.

2.Why Stack Operations are O (1)?

Stack operations like push and pop are constant time O (1) because they only add or remove elements from the top of the stack. No traversal or shifting of elements is required.

Part B: Factorial & Fibonacci (Recursion + Complexity)

Factorial Complexity

Time Complexity:
O(n) — because the function calls itself n times.

Space Complexity:
O(n) — due to recursion stack depth.

Big-Ω: O(n)
Big-Θ: O(n)

Factorial always performs exactly n recursive calls.

Fibonacci (Naive) Complexity

Time Complexity:
$O(2^n)$ — exponential growth.

Because it recomputes the same values many times.

Example: fib(30) recalculates fib(20) many times.

Space Complexity:
O(n) — recursion depth.

Big-Ω: $O(2^n)$
Big-Θ: $O(2^n)$

Fibonacci (Memoized) Complexity

Time Complexity:
O(n) — each value is computed only once.

Space Complexity:
O(n) — for recursion stack + memo dictionary.

Big-Ω: O(n)
Big-Θ: O(n)

Why Naive Fibonacci Becomes Slow

Naive Fibonacci solves the same subproblems repeatedly.
For example, fib(5) calculates fib(3) multiple times.

This repeated computation causes exponential growth in function calls. Memoization stores previously computed results, avoiding repetition. This is why memoized Fibonacci is much faster.

Part C: Tower of Hanoi (Recursion + Trace)

Why Number of Moves = $2^n - 1$

For each disk added, the number of moves doubles plus one extra move for the largest disk.

Example:

- $n = 1 \rightarrow 1$ move
- $n = 2 \rightarrow 3$ moves
- $n = 3 \rightarrow 7$ moves
- $n = 4 \rightarrow 15$ moves

Pattern:
$\text{Moves}(n) = 2 \times \text{Moves}(n-1) + 1$

Solving this recurrence gives:

$\text{Moves}(n) = 2^n - 1$

Time Complexity

Time Complexity = $O(2^n)$

Because:
Each recursive call generates two more recursive calls.

The number of operations grows exponentially.

Space Complexity

Space Complexity = $O(n)$

Reason:
The maximum recursion depth is n.
At most n function calls are on the call stack at a time.

Part D: Recursive Binary Search (Divide & Conquer + Recurrence)

Binary Search Recurrence

Binary search follows this recurrence:

$T(n) = T(n/2) + O(1)$

Because:

- Each recursive call reduces the problem size to half.
- Only constant work is done outside recursion (comparison + mid calculation).

Time Complexity

Worst Case:
O(log n) - when the element is at the last level or not present.

Average Case:
O(log n)

Best Case:
O(1) - when the element is found at the middle in the first comparison.

Space Complexity

O(log n)

Because:
The recursion depth is at most log n.

Part E: Short Paradigm Reflection

1. Binary Search – Divide & Conquer

Binary Search follows the Divide & Conquer paradigm.

In Divide & Conquer:

1. Divide the problem into smaller subproblems.
2. Solve the smaller subproblem.
3. Combine the results (if needed).

In Binary Search:

- The array is divided into two halves.
- We only continue searching in one half.
- The problem size reduces by half in every step.

This repeated halving is why the time complexity becomes O (log n).

There is no need to combine results because only one half contains the possible answer.

Therefore, Binary Search is a clear example of Divide & Conquer.

---

2. Fibonacci (Memoized) – Dynamic Programming

Memoized Fibonacci uses Dynamic Programming (DP).

Dynamic Programming is used when:

- A problem has overlapping subproblems.
- The same subproblem is solved multiple times.

In naive Fibonacci:

- fib(5) calculates fib(3) and fib(2).
- fib(3) again calculates fib(2) and fib(1).
- Many values are recomputed repeatedly.

This causes exponential time complexity.

In memoized Fibonacci:

- We store previously computed Fibonacci values.
- If a value is already computed, we reuse it.
- Each Fibonacci value is calculated only once.

This reduces time complexity from $O(2^n)$ to $O(n)$.

Therefore, memoization solves the overlapping subproblem issue, making it a Dynamic Programming approach.

---

3. Tower of Hanoi – Recursion

Tower of Hanoi is a classic example of recursion.

Recursion works by:

- Defining a base case
- Solving a smaller version of the same problem

In Tower of Hanoi:

Base Case:
If there is only 1 disk, move it directly.

Recursive Idea:
To move n disks:

1. Move $n-1$ disks to auxiliary rod
2. Move largest disk to destination
3. Move $n-1$ disks to destination

Each step solves a smaller subproblem.

The problem keeps reducing until it reaches the base case.

This demonstrates the core idea of recursion:
Breaking a big problem into smaller similar problems.

---

4. Real-Life Examples of Algorithm Paradigms

Divide & Conquer (Real-Life Example)

Searching a name in a phonebook:

- Open the book in the middle.
- If the name comes before, search left half.
- If after, search right half.
- Repeat until found.

This is similar to Binary Search.

Dynamic Programming

Budget planning:

- If you already calculated monthly expenses,
- You store the result instead of recalculating again.

Reusing previous results saves time and effort.

This is similar to memoized Fibonacci.

Greedy Algorithm

Greedy algorithms make the best immediate choice at each step.

Example:
Making change using coins.

- Always pick the largest coin first.
- Then move to the next largest.

It does not look at future consequences deeply.
It makes the locally optimal choice at each step.

References :-

1. GeeksforGeeks
Articles on: Recursive Fibonacci

Tower of Hanoi

Binary Search

Time and Space Complexity

-https://www.geeksforgeeks.org/

2. Python Software Foundation
Official Python Documentation -https://docs.python.org/3/
(Referenced for Python syntax and recursion behavior.)

4. ChatGpt for detailed information